



Implementation report: what was built and why

Fine-Tuning Neural Machine Translation for Kenyan Public Service
Announcements, adding Ekegusii to NLLB-200.

TEAM

Weldeszenbet Zera • **Samuel Abrha** • **Hetal Kumbharana** • **Halima Mohammed** • **Peter Kidiga**
• **Mitchelle Moraa**

SUPERVISOR

Prof. Edward Ombui

[---][---] | Released translation model | [samuelabrha/nllb-200-600M-ekegusii-mixed](#) | private on Hugging Face, verified | | Baseline model | [samuelabrha/nllb-200-600M-ekegusii-stage1](#) | private, verified | | Curriculum ablation | [samuelabrha/nllb-200-600M-ekegusii-psa](#) | private, verified | | Parallel corpus | [output/](#) in the repository | built, filtered, split | | Notebook pipeline | [notebooks/00](#) to [notebooks/05](#) | runs end to end | | Training script | [train_stages.py](#) | produced the released weights | | Web demo | [serve/](#) | runs locally, on a GPU node, or in Colab | | Evaluation | [artifacts/data/evaluation.json](#) | four systems, two metrics, two directions |

Everything is reproducible from the repository. Nothing depends on a machine that no longer exists.

2. The problem, stated precisely

NLLB-200 is a multilingual translation model covering just over 200 languages. **Ekegusii is not one of them.** This is not a quality gap that fine-tuning alone can close: there is no [guz_Latn](#) token in the vocabulary, no embedding for it, and therefore no way to tell the decoder "produce Ekegusii." The model cannot be asked.

About 2.7 million Kenyans speak Ekegusii, mostly in Kisii and Nyamira counties. Kenyan public service announcements are issued in English and Kiswahili, so a speaker who reads Ekegusii most comfortably receives health, tax and safety information in a second or third language, or not at all.

Two things therefore had to happen: **add the language**, then **teach it the register of a public notice**.

3. The corpus

3.1 What we assembled

No usable English–Ekegusii parallel dataset existed, so the corpus was built from scratch out of four sources.

SOURCE	RECORDS	LANGUAGES	WHAT IT GIVES
Ekegusii Revised Bible (eBible)	30,753	en · guz · swl	Verse-aligned triples, the bulk of the language signal
Kenyan PSA corpus (supervisor)	4,082	en · guz · swl	Real public notices, exactly on target. 4,082 records become 5,692 directional pairs
Lughayangu	316	en · guz	Everyday sentences, scraped from 364 pages
African Storybook	110	en · guz · swl	Contemporary prose

The English side of the Bible is the Berean Standard Bible (public domain); the Kiswahili side is Neno: Bibilia Takatifu.

3.2 What the released model actually trained on

Each aligned record can yield two directional pairs: English→Ekegusii, and Kiswahili→Ekegusii where a Kiswahili column exists. This is the table to quote.

SOURCE	DIRECTION	PAIRS
Ekegusii Bible	English to Ekegusii	28,439
Ekegusii Bible	Kiswahili to Ekegusii	28,482
Lughayangu everyday sentences	English to Ekegusii	111
Duplicates removed		-55
Kenyan PSAs	English to Ekegusii	3,509
Kenyan PSAs	Kiswahili to Ekegusii	2,183
Total unique training pairs		62,669

Held out and never trained on: **2,993** scripture pairs, **200** everyday sentences, **944** PSA pairs.

Three numbers, three meanings. Confusing them is the easiest way to be caught out:

- **~35,300 aligned records** — rows in the corpus
- **62,669 unique training pairs** — after both directions and deduplication. *This is the corpus size.*
- **79,745 examples per epoch** — the above with the 5,692 PSA pairs repeated four times

The PSA data is upsampled $\times 4$ so that 5,692 on-target pairs are not drowned by 57,000 verses of scripture. That is a training weight, not more data. [docs/count_records.py](#) recomputes all of this from the raw CSVs.

3.3 Two sources that contributed nothing

4laws supplied 27,575 English–Ekegusii pairs. Every one was a duplicate of the eBible text already in the corpus. We recorded that rather than deleting it quietly, because a source that adds zero rows is itself a finding about the state of Ekegusii resources.

African Storybook rows were merged into the Bible CSV *after* training had finished, so the released model never saw them. They are in the repository and would be included by a re-run, but they are not part of what was trained. Do not claim them. This was confirmed by replaying the split logic: 30,753 Bible rows produce exactly the 56,977 general pairs the training run reported; 30,863 produce 57,160.

3.4 Quality control

A language identifier trained on our own data. Off-the-shelf identifiers do not know Ekegusii, so a character-trigram Naïve Bayes classifier was trained on the project's own corpus. It separates English, Ekegusii and Kiswahili at 99.5–100% accuracy.

It then caught a fault in our own output: 11 of 121 African Storybook rows were not Ekegusii but licence and credits text (`* License: [CC-BY] * Text: ...`) swept up by the scraper. **A human review had already passed those rows.** That is the case for automated verification in one sentence.

Other gates, in the order they apply:

- **Mojibake repair** with `ftfy`, applied repeatedly to a fixed point, because some rows had been through a cp1252/UTF-8 round trip more than once
- **Footnote and cross-reference removal** during USFM parsing, deleting the whole span rather than the marker
- **Verse-span matching** rather than naive line alignment
- **Rejection** of empty rows, rows where Ekegusii equals English, rows the language identifier rejects, degenerate lengths, and ambiguous alignments where one Ekegusii string maps to dissimilar English

One deliberate asymmetry, and it will be questioned. Long document extracts and length-ratio outliers are **kept in training** and **excluded from the test set**. They are still Ekegusii and still teach the language, but scoring on a 200-word extract would measure document handling rather than PSA translation. The reasoning is written into `usable()` in notebook 02.

3.5 Test-set integrity

The Bible contains **421 duplicate English verses** — parallel passages where Chronicles repeats Kings. A verse could land in the test split while an identical copy sat in training, leaking the source.

Splits are deduplicated on `(source language, target language, lowercased source)`. **67 test rows and 28 dev rows** were dropped and the count is printed rather than hidden. The check is an assertion, so it runs on every rebuild.

4. The model

4.1 Transfer learning, and where the transfer happens

Base model: `facebook/nllb-200-distilled-600M`.

1. `guz_Latn` is added to the tokenizer as a special token. The embedding matrix grows from 256,204 to 256,205 rows.
2. The new row is **copied from** `kik_Latn` (Kikuyu) plus 1% Gaussian noise, so the two can diverge during training.
3. Sequences are built by hand as `[src_lang] tokens [eos]`, with `forced_bos_token_id` selecting the target. The tokenizer's own language machinery does not know about a language added after pretraining.

Step 2 is the transfer. NLLB has already learned Bantu noun-class morphology and orthography from Kikuyu, Kiswahili and others; seeding from `kik_Latn` hands Ekegusii that knowledge as a starting point. A random embedding would give the decoder no prior at all, and 62,669 pairs is nowhere near enough to learn a language from nothing.

Kikuyu is the *nearest available* Kenyan Bantu language in NLLB, not the nearest relative — Ekegusii sits in a different Bantu subgroup. Describe it as the best available proxy.

4.2 The three runs

Identical initialisation, data, hyperparameters and random seed. **Only the ordering differs.**

RUN	STARTS FROM	TRAINS ON	PURPOSE
Stage 1	tokenizer-extended base	56,977 general pairs	learn Ekegusii, the baseline
Stage 2	stage 1's weights	30,357 examples: PSA ×4 + 25% scripture replay	adapt to PSA register
Mixed	tokenizer-extended base	79,745 examples, one pass	control: did ordering help?

Replay exists because continuing training on a narrow new domain normally causes catastrophic forgetting. Mixing 25% of the old data back in is the standard defence.

4.3 Training configuration

Hardware	1 × NVIDIA A100-SXM4-80GB, shared
Precision	bf16, gradient checkpointing
Optimiser	8-bit AdamW (bitsandbytes), Adafactor fallback
Effective batch	48
Learning rate	5e-5 stage 1 and mixed, 1.5e-5 stage 2
Epochs	3, best checkpoint by dev loss
Label smoothing	0.0
Decoding	beam 4, max 128 new tokens
Seed	42, fixed across all runs

Label smoothing is off deliberately. With a 256k vocabulary the logits dominate memory, `accelerate` upcasts them to fp32, and smoothing doubles that again — roughly 500 MB per example instead of 250 MB. Turning it off is what made the batch size viable on a shared card.

Training is a script, not a notebook. `train_stages.py` waits for free VRAM, recovers from out-of-memory by halving the batch, and can resume a single stage. None of that survives a kernel restart. The training notebook was removed from the sequence so that nobody runs a second, subtly different path and reports numbers that do not match the paper.

5. Evaluation

5.1 Why chrF2++

BLEU counts whole-word n-grams. Ekegusii is agglutinative: subject, tense, object and negation attach to a verb stem, so one word carries what English spreads over five. Get the stem right and one affix wrong and BLEU scores the whole word as a miss.

chrF2++ counts character n-grams plus word unigrams and bigrams, so a correct stem with an imperfect affix earns most of the credit it deserves.

Our own numbers make the case better than the theory does. On real PSAs, English→Ekegusii:

SYSTEM	BLEU	CHRF2++
Stock NLLB-200	1.63	14.56
Stage 1, language only	2.54	23.96
Two-stage curriculum	7.21	34.93
Single pass (released)	12.33	40.97

Stage 1 demonstrably speaks Ekegusii — 49.66 chrF2++ on scripture — yet BLEU puts it at 2.54, barely above a model that cannot produce the language at all.

COMET was not used. It is a learned metric whose encoder has never seen Ekegusii, so its scores here would be noise with a decimal point.

Both metrics come from sacreBLEU. Bootstrap confidence intervals are reported.

5.2 Results

chrF2++, four systems, both directions.

DIRECTION	TEST SET	N	STOCK	STAGE 1	TWO-STAGE	RELEASED
eng→guz	Real PSAs	570	14.56	23.96	34.93	40.97
eng→guz	Scripture	1,459	14.88	49.66	48.95	49.81
eng→guz	Everyday prose	200	11.41	28.97	29.37	32.98
swh→guz	Real PSAs	371	14.13	24.49	34.50	39.61
swh→guz	Scripture	1,463	14.65	49.11	48.80	49.30

BLEU, same layout:

DIRECTION	TEST SET	STOCK	STAGE 1	TWO-STAGE	RELEASED
eng→guz	Real PSAs	1.63	2.54	7.21	12.33
eng→guz	Scripture	0.83	19.75	19.33	19.91
eng→guz	Everyday prose	0.07	1.27	1.21	1.59
swh→guz	Real PSAs	1.01	1.77	6.87	10.66
swh→guz	Scripture	1.20	19.79	19.79	19.97

Stock NLLB cannot produce Ekegusii; it is asked for Kikuyu, so its column is a **floor rather than a baseline**. Everyday prose has no Kiswahili row because the Lughayangu corpus is English–Ekegusii only.

5.3 What the numbers say

- **+26.41 chrF2++ over the floor** on real PSAs, a 181% relative gain. On BLEU, 7.6× the floor.
- **+17.01 over stage 1** — this is the value of PSA adaptation specifically.
- **Zero forgetting**. The released model scores 49.81 on scripture, higher than the model trained only on scripture.
- **Both directions agree**. Kiswahili→Ekegusii gains 180% against English's 181%, so the result is not an artifact of one source language.

5.4 The curriculum lost

The two-stage curriculum scored **34.93** on real PSAs; the single-pass control scored **40.97**. Curriculum benefit is **−6.04 chrF2++**.

Both easy objections are ruled out by construction:

- **Not more data**. The single pass sees every unique example the curriculum sees. The replay slice is duplicated scripture already present in stage 1.
- **Not more compute**. At equal epochs the two-stage run takes roughly **9% more gradient updates**, because replayed rows are trained on twice. It had more compute and still lost.

A plausible mechanism: stage 2's dev loss bottomed at 1.357 and drifted back to 1.417, consistent with overfitting a small PSA-heavy set. The single pass sees PSAs interleaved with scripture throughout, which regularises it. And the forgetting that replay was designed to prevent never arises when the data is not split.

We release the control. A hypothesis that survives a test it could have failed is worth more than one never tested.

6. Verification

Three checks, all scripted and repeatable.

Model identity. `tests/identify_hf_models.py` compares each published repository against its local checkpoint by weight fingerprint: cosine similarity in float64 over a fixed tensor slice, plus the `guz_Latn` embedding row, which diverges fastest between runs. Every repository matched its own directory at **1.000000**; every cross-pair sat at 0.99985–0.999985. Filenames prove nothing; tensors do.

Behavioural confirmation. Re-scoring 150 held-out PSA rows with each downloaded model reproduced the ordering and approximate values: 23.77 / 36.53 / 43.55 against reported 23.96 / 34.93 / 40.97.

Upload integrity. `tests/verify_hf_uploads.py` checks each repository has weights, config and tokenizer, and that the published tokenizer resolves `guz_Latn`. This catches the quiet failure where weights upload without the tokenizer: the repository looks healthy, downloads without error, and cannot produce Ekegusii.

7. Deployment

`serve/` holds one FastAPI application that runs in two shapes.

MODE	ENABLED_SYSTEMS	WHAT YOU GET
Public demo	<code>mixed</code>	One model, direction picker, confidence label, correction form
Internal comparison	<code>stock,stage1,stage2,mixed</code>	Four models side by side plus the chrF chart

Three ways to run it:

```
# on a GPU node, public URL via Cloudflare tunnel
bash serve/run_public_demo.sh
```

```
# locally with Docker
cd serve && docker compose --profile cpu up --build
```

```
# from Google Colab, free T4, four cells
serve/colab_demo.ipynb
```

The confidence label is the model's own certainty, the geometric-mean per-token probability of the output it chose. It is *not* a probability of being correct, the band thresholds are uncalibrated, and the interface says so. Calibrating them requires the human evaluation we do not yet have.

The correction form is the most valuable thing the deployment produces. Set `FEEDBACK_REPO` to a Hugging Face dataset repository or corrections are written to a container disk that gets wiped. Every correction is both the missing human evaluation and a training pair for the next round.

The service is rate-limited to 30 requests per minute per client IP, because a public URL in front of a GPU is an open invitation otherwise. Behind a tunnel every request appears to come from 127.0.0.1, so the limiter reads `CF-Connecting-IP`.

8. Repository map

notebooks/	00 setup · 01 EDA · 02 data · 03 tokenizer · 04 eval · 05 export
_removed/	two notebooks dropped, kept for reference
train_stages.py	the training path that produced the released weights
nb_common.py	shared paths, palette, seeds, tokenizer helpers
ekegusii/	corpus builders: Bible aligner, PSA preparation, scrapers
serve/	the web service, Docker files, Colab notebook
tests/	notebook validator, upload verifier, model fingerprinter
docs/	deck, banner, this report, presentation brief, generators
output/	the corpus CSVs
artifacts/	models, splits, figures, metrics (not committed)

Run the sequence in order: `00_setup` → `01_eda` → `02_build_training_data` → `03_extend_tokenizer` → `train_stages.py` → `04_evaluate` → `05_inference_and_export` .

Two notebooks were removed. `02_retranslate_swahili` was dead: Kiswahili is a source language here and never a target, so nothing downstream read its output. `05_finetune` never trained the released weights and was a second, subtly different path to the same thing.

9. Known limitations and open work

No human evaluation. Every number is an automatic metric against a single reference. No fluency, adequacy or cultural-accuracy ratings from Ekegusii speakers exist. This is the largest gap and the demo's correction form is the mechanism for closing it. The brief asks for 100+ rated sentences.

References are unaudited. The PSA Ekegusii was supplied by the supervisor, who speaks the language, and one member of the team speaks it too. But the translator behind the corpus and the quality-assurance process that produced it are unrecorded, no one has re-checked it end to end, and there is no inter-annotator agreement. The capability to fix this is in the room, which is what makes leaving it undone the wrong call.

Scripture-heavy mixture. About 57,000 of 62,669 pairs are Bible verses, so the model may lean formal on unfamiliar input.

Institutional vocabulary. 53.6% of English content-word types in the PSA corpus (42.3% of tokens) never appear in the training data. HELB, KUCCPS, iTax and similar remain the weakest cases. This was measured, not solved.

Licensing. The weights derive from the Ekegusii Revised Bible, © Bible Society of Kenya. The repositories are private pending clearance. Do not make them public without checking.

Hosting is temporary. Hugging Face Spaces now require a paid plan for Docker and CPU Gradio Spaces; ZeroGPU needs an account older than 30 days or a community grant. Until one of those lands, the demo runs from Colab or a tunnel and the URL changes each time.

Next steps, in priority order

1. Collect human evaluation through the correction form
2. Request a Hugging Face community grant for permanent hosting
3. Back-translate monolingual Ekegusii text to enlarge the corpus
4. Target institutional vocabulary directly, possibly with a terminology list
5. Re-run the recipe on NLLB-200-1.3B now that it is settled

10. Reproducing this from zero

```
git clone https://github.com/SamAbr/PSA-MT.git
cd PSA-MT
pip install -r requirements.txt
```

```
python ekegusii/prepare_psa_ke.py          # merge and filter the PSA corpora
jupyter lab                                # run notebooks 00 to 03
python train_stages.py --stages 1 2 mixed --min-free 12 --wait-mins 720
# then notebooks 04 and 05
```

Notebooks fetch their data from GitHub over HTTPS, so a bare GPU node needs only the repository. Expect 8–13 hours for all three training runs and 45–90 minutes for evaluation.

One caveat on exact reproduction. The Bible CSV now contains 110 African Storybook rows that were added after training. A re-run will therefore produce 57,160 general pairs rather than 56,977, and results will differ slightly. To reproduce the released model exactly, filter `bible_en_guz_swh.csv` to `source == "bible_ebible"` first.